

PicoGPT

PicoGPT: Implementacija modifikovane "nanoGPT" arhitekture od nule korišćenjem biblioteka NumPy i CuPy.

Definicija Problema

Moderno duboko učenje se u velikoj meri oslanja na softverske okvire (kao što su PyTorch ili TensorFlow) koji kriju matematičku složenost optimizacije i propagacije greške unatrag iza automatizovanih mehanizama (autograd engine). Ovaj pristup, iako efikasan, predstavlja "crnu kutiju" koja ograničava striktnu kontrolu nad proračunima, onemogućava laku adaptaciju modela na različitim hardverskim arhitekturama i redukuje mogućnost detaljnog razumevanja unutrašnjih mehanizama velikih jezičkih modela (LLM).

Implementacijom generativnog transformera (Transformer) isključivo kroz čiste matematičke operacije nad matricama korišćenjem biblioteke NumPy, ovaj projekat razvija potpuno transparentan, auditable (proverljiv) i nezavisan kod za GPT modele. Praktična primenljivost ovakvog rešenja ogleda se u mogućnosti izvršavanja i obučavanja jezičkih modela u visoko ograničenim "edge" okruženjima ili na specifičnom namenskom hardveru gde teški radni okviri za duboko učenje (sa stotinama zavisnosti i složenim grafovima) ne mogu biti instalirani ili kompajlirani. Projekat pruža duboki uvid u minimalne matematičke zahteve neophodne za funkcionisanje ovih modela.

Skup podataka

U okviru ovog projekta biće korišćen sledeći skup podataka u podeli 90% za trening i 10% za validaciju.

TinyStories ([Link do skupa podataka](#)) - sadrži 2 miliona redova, sa prosečno 900 karaktera po redu

Prema istraživanju Eldan i Li (2023), ovaj skup podataka sadrži kratke priče generisane upotrebom jednostavnog rečnika koji obično koriste trogodišnja deca. Dokazano je da se nad ovim skupom mogu uspešno obučavati modeli sa manje od 10 miliona parametara (pa čak i sa samo jednim transformer blokom), a da pritom zadrže visoku gramatičku fluentnost, konzistentnost kroz nekoliko pasusa i elementarne sposobnosti zaključivanja. Ovo je ključno za naš projekat usled hardverskih ograničenja (obučavanje na CPU kroz NumPy i GPU sa malo memorije pomoću CuPy).

Opis atributa i ciljnog obeležja

S obzirom na to da razvijamo autoregresivni jezički model, sirovi tekstualni podaci se transformišu u nizove celih brojeva pomoću tokenizatora na nivou karaktera (Character-level tokenizer).

- Ulazne instance (X): Matrica celobrojnih tokena dimenzija (B,T), gde je B veličina paketa (batch size), a T kontekstni prozor (context/block size). Svaki element predstavlja indeks karaktera u rečniku.
- Ciljno obeležje (Y): Matrica identičnih dimenzija (B,T), ali pomerena za jedno mesto udesno u odnosu na ulazni niz ($Y_t = X_{t+1}$). Sadržaj ciljnog obeležja je indeks sledećeg karaktera u nizu koji model treba da predvidi.
- Najznačajniji atributi: Kontekstni prozor prethodnih karaktera na osnovu kojih se računa uslovna verovatnoća sledećeg tokena. Rečnik (V) se sastoji od približno 65 jedinstvenih karaktera (slova, brojevi, novi red...).

Metodologija

Arhitektonski elementi rešenja

1. Strukturalni elementi i raspored matrica

Model će manipulirati hiper parametrima prilagođenim radu na CPU: Veličina rečnika $V \approx 65$, kontekst $T=64$, embedding dimenzija $C=128$, broj blokova $L=3$, broj pažnji unutar bloka $NH=4$ (dimenzija svake glave je $HS=C/NH=32$).

Model će biti agnostic što znači da može da se pokrene i na CPU i na GPU ako je dostupan

2. Detaljan pregled ključnih modula

- Causal Multi-Head Attention (MHA) sloj: Ulazna matrica skrivenih stanja X oblika (B,T,C) se množi sa težinama za projekciju kako bi se dobili tenzori upita (Q), ključeva (K) i vrednosti (V). Ovi tenzori se dele na glave, poprimajući oblik (B,NH,T,HS). Računanje težina pažnje se vrši preko skaliranog proizvoda:
- Primenjuje se mehanizam uzročnog maskiranja (Causal Masking) gde se gornja polovina matrice (budući tokeni) postavlja na $-\infty$. Nakon primene Softmax funkcije, vrši se mehanizam agregacije vrednosti. Na kraju se vrši spajanje glava nazad u oblik (B,T,C) i linearna projekcija.
- Rotary Position Embeddings (RoPE): Umesto dodavanja apsolutnih pozicionih vektora na samom početku mreže, primenjuje se RoPE (Su et al., 2021) unutar same GQA strukture. RoPE rotira dvodimenzionalne isečke vektora unutar Query i Key tenzora za ugao koji zavisi od njihove pozicije m u nizu. Ova operacija se izvršava nad svakom glavom pojedinačno za Q i K pre skaliranog proizvoda, implementirajući relativne pozicije odnose direktno u geometrijski prostor pažnje
- SwiGLU aktivacioni i gating mehanizam: Umesto standardne GELU ili ReLU funkcije, implementiran je SwiGLU (Swish Gated Linear Unit) mehanizam

(Shazeer, 2020). Ulazni tenzor se projektuje na dimenziju (B, T, 2 x dffn), a zatim polovi duž poslednje ose na dve komponente: X1 i X2. Prva polovina prolazi kroz SiLU (Swish) aktivaciju, te se elementarno množi (Hadamardov proizvod) sa drugom polovinom

- RMSNorm (Root Mean Square Normalization): Umesto klasične Layer klase normalizacije, uvodi se RMSNorm (Zhang i Sennrich, 2019). RMSNorm stabilizuje unutrašnja stanja mreže tako što normalizuje elemente isključivo na osnovu njihovog korena srednjeg kvadrata duž dimenzije C

3. Plan izvršavanja Backpropagation postupka

Unazadni prolaz se izvodi analitički za svaki sloj koristeći pravilo izvoženja složenih funkcija (Chain Rule):

- Gubitak do logita (Loss to Logits): Izvod Cross-Entropy funkcije gubitka u odnosu na izlazne logite Z svodi se na jednostavnu razliku: $\partial Z \partial L = P - Y$, gde je P matrica softmax verovatnoća, a Y matrica one-hot targeta.
- Propagacija kroz GQA i Redukcija Gradijenata: Tokom unazadnog prolaza kroz GQA, gradijenti koji se vraćaju kroz emitovane (broadcasted) Key i Value glave moraju biti akumulirani (sumirani) duž odgovarajućih grupa glava. Ako G Query glava deli jednu KV glavu, gradijent za tu KV glavu je suma gradijenata iz svih G parova unutar te grupe
- Diferenciranje kroz RoPE rotaciju: Pošto je RoPE ortogonalna transformacija (rotaciona matrica), unazadni prolaz kroz RoPE za tenzore Q i K se rešava primenom inverzne rotacije (što je ekvivalentno rotaciji za ugao -m theta) nad uzvodnim gradijentima $\partial L / \partial Q$ i $\partial L / \partial K$
- Propagacija kroz SwiGLU i RMSNorm: Unutar MLP bloka, primenjuje se pravilo proizvoda (Product Rule) za pronalaženje pojedinačnih gradijenata $\partial L / \partial X1$ i $\partial L / \partial X2$, nakon čega se vrši njihovo spajanje (concatenation). Gradijent kroz RMSNorm se računa bez potrebe za praćenjem srednje vrednosti, što pojednostavljuje tenzorske parcijalne izvode.

4. Optimizacioni mehanizam i strategija treniranja

- a. Izbor optimizatora: Biće implementirani AdamW optimizator u čistom NumPy-ju koji prati prvi (m) i drugi (v) momenat gradijenata za sve parametre, vrši korekciju pristrasnosti i adaptivno menja stopu učenja (learning rate).
- b. Verifikacija: Tokom razvoja, inicijalizovaće se paralelni, identični model u PyTorch frejmworqu sa istim težinama. Nakon jednog forward i backward prolaza, vršiće se poređenje gradijenata izračunatih u NumPy-ju sa onima iz PyTorch-a uz dozvoljenu numeričku toleranciju (Gradient Check, npr. $\Delta < 10^{-6}$)

Koraci realizacije projekta

1. Cevovod podataka i tokenizacija (Data Pipeline & Tokenization):
 - a. Preuzimanje i parsiranje tekstualnih fajlova (TinyStories).
 - b. Izrada kastomizovane CharacterTokenizer klase sa metodama encode i decode.
 - c. Razvoj BatchLoader generatora koji iz tekstualnog niza nasumično izdvaja i strukturira ulazne matrice X i pomerene ciljne matrice Y dimenzija (B,T).
2. Podešavanje referentnog okruženja (Reference Framework Setup):
 - a. Kreiranje ekvivalentnog mini-GPT modela u PyTorch-u sa ugrađenim ekvivalentima za RMSNorm i SwiGLU topologije radi validacije.
3. Razvoj Forward Pass-a:
 - a. Inicijalizacija orišćenjem modifikovane Xavier/Kaiming raspodele. Težine unutar rezidualnih blokova biće skalirane kako bi se sprečilo zasićenje gradijenata u dubokim mrežama
 - b. Kodiranje osnovnih NumPy slojeva (Embedding, Dense, RMS, SwiGLU).
 - c. Implementacija RoPE funkcije za 4D tenzore (primena trigonometrijskih matrica na parove dimenzija).
 - d. Razvoj GQA sloja sa logikom grupisanja i uzročnog maskiranja.
 - e. Ulančavanje svih slojeva u glavnu klasu i verifikacija izlaznih logita sa PyTorch-om.
4. Backward Pass & Gradient Verification:
 - a. Pisanje eksplicitnih .backward() metoda za svaki sloj na osnovu parcijalnih izvoda matrica.
 - b. Sprovođenje automatskog testa provere gradijenata poređenjem NumPy matrica gradijenata sa PyTorch .grad atributima.
5. Petlja za treniranje i zaključivanje:
 - a. Integracija Adam optimizatora i kreiranje glavne petlje koja vrši optimizaciju nad mini-paketima podataka kroz Cross-Entropy gubitak.
 - b. Razvoj autoregresivne petlje za generisanje teksta koja primenom temperature/softmax-a generiše nove karaktere jedan po jedan.
 - c. Gradient clipping
 - d. Checkpointing

Način evaluacije

Rezultati modela i uspešnost procesa obučavanja evaluiraće se kroz dve metrike:

- Kvantitativna evaluacija: Primarna metrika je Cross-Entropy gubitak (Negativna log-verovatnoća) računata na validacionom podskupu koji model nije video tokom treninga. Cilj je pratiti opadanje vrednosti gubitka kroz iteracije, što potvrđuje da model uspešno uči statističku strukturu teksta.

- Kvalitativna evaluacija: Generisanje uzoraka teksta (Generative text sampling) u pravilnim vremenskim intervalima. Uspešnost projekta se vizuelno i lingvistički evaluira kroz analizu sposobnosti modela da pređe iz faze generisanja nasumičnih karaktera u fazu gde ispravno formira reči, razmake, interpunkciju i generiše pasuse koji oponašaju strukturu bajki (TinyStories). Generisanje će podržavati temperaturno skaliranje (temperature control) za kontrolu kreativnosti modela.

Tehnologije

- Programski jezik: Python
- Osnovna biblioteka za proračune (Core Engine): NumPy (za sve forward i backward operacije, manipulaciju matricama i optimizaciju) odnosno CuPy za izvršavanje na GPU
- Pomoćna biblioteka za podatke: Pandas (za opciono mapiranje metapodataka i skladištenje metrika)
- Referentni okvir (Oracle Okruženje): PyTorch (isključivo za automatsku verifikaciju tačnosti gradijenata na hardveru)
- Vizuelizacija rezultata: Matplotlib (za plotovanje krive gubitka kroz epohe)

Primeri gotovih rešenja

- nanoGPT (Andrej Karpathy): <https://github.com/karpathy/nanogpt> - Zvanična, optimizovana PyTorch implementacija koja služi kao glavna inspiracija za arhitekturu i hipeparametre.
- microGPT (Andrej Karpathy): <https://karpathy.github.io/2026/02/12/microgpt/> - Minimalistički osvrt na svedene arhitekture jezičkih modela
- transformer-from-scratch (DorsaRoh): <https://github.com/DorsaRoh/transformer-from-scratch> - Edukativni repozitorijum sa primerom implementacije transformera sa fokusom na bazične operacije.
- LLaMA Reference Implementation: Osnova za strukturalnu modifikaciju slojeva normalizacije i aktivacije koji su implementirani unutar ovog projekta.
- mistral-src (Mistral AI): Referentni kod za implementaciju Grouped-Query Attention mehanizma na mikro-skali.

Relevantna literatura

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention is all you need*. <https://arxiv.org/abs/1706.03762>

Zhang, B., & Sennrich, R. (2019). *Root mean square layer normalization*. Advances in Neural Information Processing Systems, 32. <https://arxiv.org/abs/1910.07467>

Shazeer, N. (2020). *GLU variants improve transformer*. arXiv preprint arXiv:2002.05202. <https://arxiv.org/abs/2002.05202>

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2021). *RoFormer: Enhanced transformer with rotary position embedding*. arXiv preprint arXiv:2104.09864. <https://arxiv.org/abs/2104.09864>

Shazeer, N. (2019). *Fast transformer decoding: One write-head is all you need*. arXiv preprint arXiv:1911.02150. <https://arxiv.org/abs/1911.02150> (*Rad koji je uveo Multi-Query Attention*).

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., León, F., & Arora, S. (2023). *GQA: Training generalized multi-query transformer models from 1 billion parameters*. arXiv preprint arXiv:2305.13245. <https://arxiv.org/abs/2305.13245> (*Rad koji je uveo Grouped-Query Attention*).

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. <https://www.deeplearningbook.org/>

Eldan, R., & Li, Y. (2023). *TinyStories: How small can language models be and still speak coherent English?* <https://arxiv.org/abs/2305.07759>

Karpathy, A. (2023). *Neural Networks: Zero to Hero Video Series*. YouTube.